

The Runner-Up Relay Route

Every spring, the trail-running guild of Fennwick charts the great relay between the clubhouse at station 1 and the summit lodge at station n . The n stations are joined by m two-way trails, trail i linking stations u_i and v_i with a running time of w_i seconds, and the trail map keeps every station reachable from every other.

A relay route starts at station 1, ends at station n , and may pass through any stations and trails along the way — runners are even allowed to double back and reuse a trail or revisit a station if they wish. The total time of a route is the sum of the running times of every trail crossing made, counted once per crossing.

The guild's champion already holds the record for the fastest possible total time. To keep the rivalry alive, the vice-champion wants to chase a different mark: the smallest total time that is strictly greater than the fastest possible time.

Given the trail map, find that runner-up time — the second-smallest distinct total time over all possible relay routes from station 1 to station n .

Function Description

Implement the function `findSecondShortestTime`.

Parameters

- n : the number of stations.
- m : the number of trails.

```
31
32
33
d2[v]=d1[v];
d1[v]=next_d;
pq.push({d1[v], v});
```

Test cases Quick test Results

Accepted 10/10 testcases passed

Score 50 Time taken 0.87061 s Memory used 39944 KIB

Your highest scored submission will be selected as the best submission

Test cases	Time (s)	Memory (KIB)	Score	Result
Test case 1	0.009155	2	2.5	Accepted
Test case 2	0.009023	2	2.5	Accepted
Test case 3	0.017345	2	5	Accepted
Test case 4	0.065733	20956	5	Accepted
Test case 5	0.114789	31944	5	Accepted
Test case 6	0.131079	35604	5	Accepted
Test case 7	0.14718	39492	5	Accepted
Test case 8	0.114623	39264	5	Accepted
Test case 9	0.130952	39944	7.5	Accepted
Test case 10	0.130735	39084	7.5	Accepted

Section: Coding Programming

50.00



Calibration Console of the Old Mill

Deep in the cellar of the Old Mill of Bramwick sits a brass calibration console with a single dial display and three worn buttons. The display always shows a whole number between 1 and n , and every calibration session begins with the display showing exactly 1.

The three buttons behave rigidly. The first increases the displayed number by 1, the second decreases it by 1, and the third multiplies it by a fixed gear factor b . The console jams if the display would ever leave the range 1 to n , so the millwright never presses a button that would make the number less than 1 or greater than n .

Every gauge in the mill has a serial mark from 2 to n , and each gauge must be calibrated once. Calibrating the gauge with mark k means starting a fresh session at 1 and pressing buttons until the display shows exactly k . The millwright always uses the minimum possible number of button presses for each gauge.

The mill's ledger requires the total wear on the console. This is the sum, over all marks k from 2 to n , of the minimum number of button presses required to reach k from 1.

Compute this total.

Function Description

Implement the function `computeCalibrationCost`.

Parameters

< Previous

2/2

Next >

C++17 (g++ 10.3.0)

```
1 > #include <bits/stdc++.h> ...
4 long long computeCalibrationCost (int n, int b) {
5     // Write your code here
```

Test cases Quick test Results

Accepted 10/10 testcases passed

Score 50 Time taken 0.08994 s Memory used 2 KiB

Your highest scored submission will be selected as the best submission

Test cases	Time (s)	Memory (KiB)	Score	Result
Test case 1	0.00918	2	2.5	Accepted
Test case 2	0.008978	2	2.5	Accepted
Test case 3	0.008961	2	5	Accepted
Test case 4	0.009177	2	5	Accepted
Test case 5	0.009007	2	5	Accepted
Test case 6	0.008907	2	5	Accepted
Test case 7	0.008924	2	5	Accepted
Test case 8	0.00909	2	5	Accepted
Test case 9	0.008782	2	7.5	Accepted
Test case 10	0.008932	2	7.5	Accepted